

Software Architecture and Design Specification

Project: Personal Finance & Expense Manager (PFEM)
Release: 1.0 Version: 1.0

Team: Team 9
Date: 02-10-2026
Status: Draft

Document property	Value
Primary baseline	PFEM Software Requirements Specification (SRS)
Testing baseline	PFEM Test Plan
Design template	Software Architecture and Design Specification template
Architecture style	Three-tier layered client-server architecture
Primary technology direction	MERN-aligned web/mobile application with REST API

This document translates the approved SRS into an implementable architecture and design. It does not redefine product requirements.

Revision History

Version	Date	Author	Change Summary
1.0	02-10-2026	Team 9	Initial Software Architecture and Design Specification based on the PFEM SRS and Test Plan.

Approvals

Role	Name	Signature / Date
Project Team	Team 9	
Faculty / Evaluator		

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) defines the architecture, major components, design decisions, interfaces, security controls and principal interaction flows for the Personal Finance & Expense Manager (PFEM), Release 1.0. The SRS is the agreed requirements baseline; this document describes how those requirements are organized into software components and interactions.

1.2 Scope

The design covers the end-user responsive web/mobile clients, the browser-based administration console, the REST application tier, domain services, scheduler, persistent data layer, notification and email integrations, private object storage for receipt attachments, and currency exchange-rate integration. Bank/card/UPI aggregation, investment tracking, tax computation, loan/EMI amortisation, shared ledgers and financial-product recommendation remain outside Release 1.0.

1.3 Audience

- Developers: component boundaries, interfaces, data ownership and implementation guidance.
- QA engineers: design-to-test traceability and expected interaction boundaries.
- Security reviewers: authentication, authorization, audit, data protection and threat controls.
- Faculty/evaluators: architecture rationale, UML views and requirement traceability.
- Maintenance teams: modularity, logging, deployment and future extension points.

1.4 Definitions

Term	Definition
PFEM	Personal Finance & Expense Manager.
REST API	HTTP-based application interface used by the clients to invoke application services.
RBAC	Role-based access control; permissions are enforced according to the authenticated role.
Scheduler	Application-tier component that processes recurring rules, reminders and scheduled maintenance jobs.
Soft delete	Logical deletion in which data remains recoverable for the defined retention period.
SAD	Software Architecture and Design Specification.
SRS	Software Requirements Specification.
RTM	Requirement Traceability Matrix.

2. Document Overview

2.1 How to Use This Document

Sections 3 and 4 are the main implementation references. Section 3 establishes architectural goals, stakeholders, the component structure, architectural pattern, technology/data choices, risks, traceability and security architecture. Section 4 provides detailed interaction flows, API contracts, error/logging conventions, UX constraints and open issues. The appendices summarize terminology, references and design decisions.

2.2 Related Documents

- Personal Finance & Expense Manager — Software Requirements Specification (SRS), Release 1.0.
- Personal Finance & Expense Manager — Test Plan, containing UT, IT and ST cases and coverage.
- PFEM SRS Appendix C — Requirement Traceability Matrix.
- Project deliverable/template document for Software Architecture and Design Specification.

2.3 Architectural Drivers

Driver	Design implication
Financial-data confidentiality	All user data access is mediated by the application tier and authorization checks.
Fast transaction entry	Thin client, focused REST transaction endpoint and indexed database queries.
Derived financial figures	Domain services centralize balance, budget, goal and reporting computations.
Scheduled behavior	Scheduler runs in the application tier for recurring entries, reminders, rollover and purge tasks.
Data portability	Dedicated import/export services isolate CSV/JSON/PDF processing from core transaction logic.
Recoverability	Soft deletion, atomic operations and encrypted daily database backups.
Maintainability	Layered modules with reusable authentication, notification, upload and CSV-processing modules.

3. Architecture

3.1 Goals & Constraints

Goals. Separate presentation, business logic and persistence; protect financial data; support responsive web/mobile clients; keep scheduled processing reliable; provide reusable domain-independent services; meet the response-time targets defined by the SRS; and make the design testable through clear component boundaries.

Constraints. The SRS specifies a three-tier client-server architecture, a document database, application-layer schema validation, REST-style external communication, and MERN-aligned implementation. The Release 1.0 boundary deliberately excludes direct bank/payment-network integration and several advanced financial features.

3.2 Stakeholders & Concerns

Stakeholder	Primary concerns
End user	Correct balances, quick transaction entry, privacy, useful reports, reliable reminders and recoverable deletion.
Administrator	User/account administration and audit visibility without access to users' financial figures.
Development team	Modularity, clear interfaces, reusable modules, deployment repeatability and maintainability.
QA team	Requirement traceability, deterministic validation, atomic operations and observable failures.
Security reviewer	Authentication, authorization, password protection, auditability, data isolation and secure attachment access.
Faculty / evaluator	Traceability from SRS requirements to architecture, design and tests.

3.3 Component (UML) Diagram

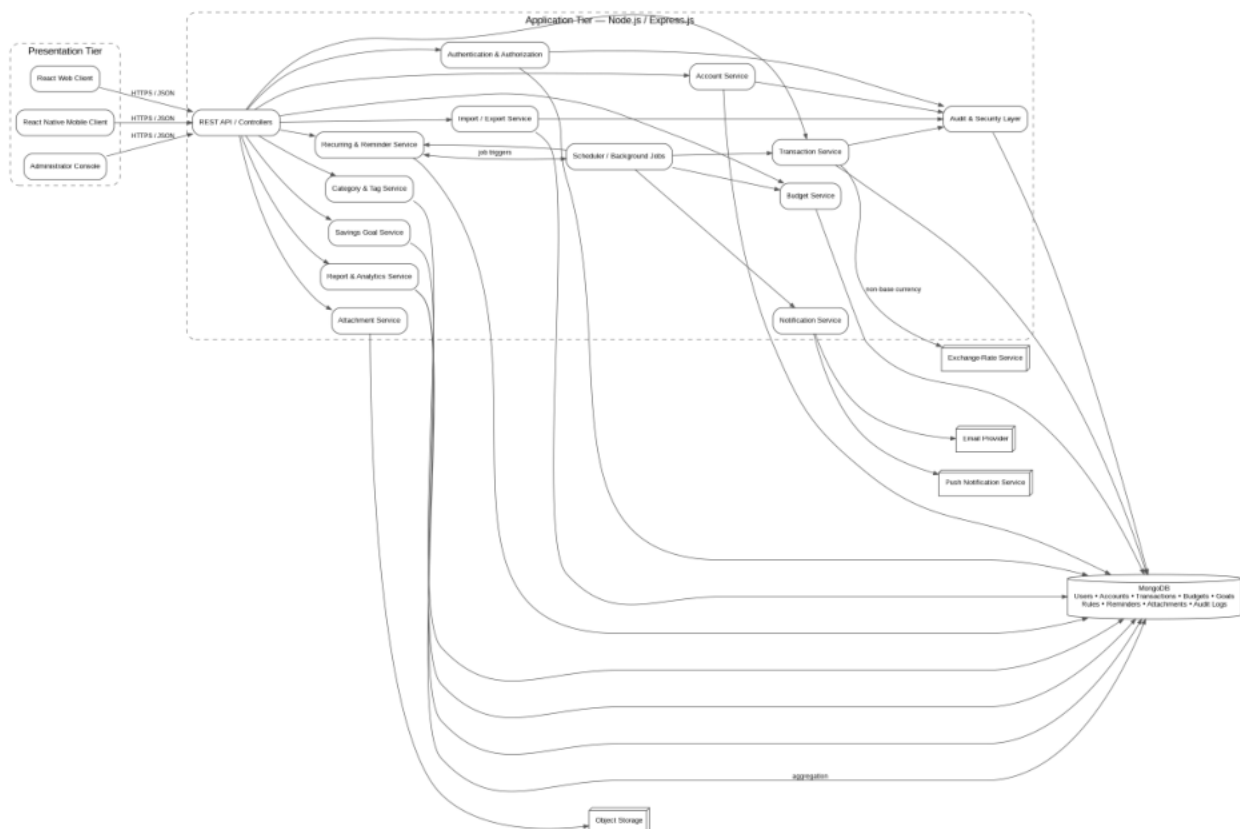


Figure 3.1 — PFEM component architecture.

The presentation tier contains the responsive web client, mobile client and administrator console. The application tier exposes the REST API and contains authentication, transaction/account, category, budget, recurring/reminder, goal, reporting, import/export, attachment, notification, security and scheduler responsibilities. Only the application tier accesses the data tier. External integrations are isolated behind service adapters.

3.4 Component Descriptions

Component	Responsibility
Web / Mobile Client	User interaction, local presentation state, validation feedback, dashboard/report rendering and authenticated API calls.
Admin Console	Administrative operations and audit-log views subject to RBAC.
REST API Layer	Routing, request validation, authentication middleware, response formatting and error mapping.
Auth Service	Registration, email verification, login, password recovery/change, session/JWT handling and lockout rules.
Transaction & Account Services	Expense/income/transfer processing, account balances, transaction edits and recycle-bin behavior.
Budget Service	Budget creation, overlap validation, utilization, threshold evaluation and rollover.
Recurring / Reminder Services	Recurring rule lifecycle, due transactions, bill reminders and overdue handling.
Goal Service	Savings-goal lifecycle, contributions, balance validation and achievement state.
Report Service	Dashboard summaries, category breakdowns, trends, comparisons, projections and net-worth calculations.
Import / Export Service	CSV validation/preview/commit and CSV/JSON/PDF export.
Attachment Service	Receipt validation, object-storage upload and signed retrieval URLs.
Notification Service	Email/push dispatch and delivery status handling.
Scheduler	Recurring jobs, reminders, rollover, soft-delete purge and other scheduled maintenance.
Data Access Layer	MongoDB access, indexes, application-level referential consistency and aggregation queries.
Audit / Security Layer	RBAC enforcement, permission checks, audit records and security-related cross-cutting controls.

3.5 Chosen Architecture Pattern and Rationale

PFEM uses a layered three-tier architecture. Presentation concerns are separated from application/domain logic, which is separated from persistence. This directly follows the SRS product perspective and provides a manageable structure for a four-member software-engineering project.

Layer	Responsibility	Allowed dependency direction
Presentation	Web/mobile/admin UI, client-side state, rendering and user feedback.	Calls application APIs; does not access MongoDB directly.
Application / Domain	REST endpoints, validation, business rules, calculations, scheduling, notifications and orchestration.	May call data-access modules and external-service adapters.
Data	Persistent PFEM documents, indexes, aggregation and backup/restore infrastructure.	Accessible only through the application tier.

Microservices are not selected as the primary decomposition for Release 1.0 because the SRS calls for a single application tier and the project requires a controlled, maintainable implementation. The internal services are therefore modular components within the application tier rather than separately deployed services.

3.6 Technology Stack & Data Stores

Area	Technology / Design
Web client	React-based responsive client, following the SRS UI requirements.
Mobile client	Mobile client aligned with the SRS mobile requirements and shared REST API.
Application server	Node.js with Express-style REST API, consistent with the MERN requirement.
Database	MongoDB document database; application-layer schema validation and indexed queries.
Authentication	Password hashing using bcrypt (work factor ≥ 12 per SRS), email verification, JWT sessions and account lockout.
Attachments	Private object storage with short-lived signed URLs.
Email	SMTP or provider REST API.
Push	Push-notification service through its current SDK/API.
Currency conversion	External exchange-rate REST API only for non-base-currency transactions; used rate is stored.
Charts	Reusable client-side charting library as required by the SRS.
Transport	HTTPS for client/API communication; secure provider connections for external services.

3.7 Risks & Mitigations

Risk	Mitigation
Incorrect balance after transaction failure	Atomic transaction/transfer handling; operations either complete fully or leave financial data unchanged.
Unauthorized cross-user access	Authentication plus request-level RBAC/ownership checks; user identifier included in data-access predicates.
Credential compromise	Salted bcrypt hashes, JWT expiration, lockout after repeated failed sign-ins and password-change verification.
Attachment exposure	Private object storage and short-lived signed URLs generated only after ownership checks.
Scheduler duplication or outage	Idempotent due-date processing, original-date preservation for missed jobs and duplicate prevention.
Data loss	Encrypted daily backups, seven-day retention and restoration testing.

Risk	Mitigation
Slow reporting queries	Indexed user/date/category queries and aggregation pipelines; performance targets monitored against SRS thresholds.
Import corruption	Preview/validation stage and atomic commit of valid rows only.

3.8 Traceability to Requirements

The complete requirement-to-test mapping remains in the SRS Appendix C and Test Plan. The following design traceability highlights representative requirements and the architecture/design element responsible for them.

SRS requirement	Architecture reference	Design reference / implementation responsibility
REQ-1-REQ-8	Auth Service	Registration, verification, password policy, JWT and logout flow.
REQ-16-REQ-24	Transaction + Account Services	Transaction model, validation and transfer atomicity.
REQ-29	Transaction Service + Scheduler	Recycle-bin retention and purge job.
REQ-36-REQ-42	Category + Report Services	Category model, delete cascade and aggregation pipeline.
REQ-43-REQ-48	Budget Service + Scheduler	Budget model, overlap validation, thresholds and rollover.
REQ-52-REQ-60	Recurring/Reminder Services + Scheduler	Recurring model, due-job processing and reminder flow.
REQ-61-REQ-66	Goal + Account Services	Goal model, contribution transaction and achievement state.
REQ-69-REQ-78	Report Service + Client	Dashboard query, chart data contract and export design.
REQ-79-REQ-86	Attachment + Import Services	Upload validation, CSV pipeline and post-import recomputation.
REQ-89	Auth Service	Password-change flow with current-password verification.
REQ-93	Security Layer	RBAC middleware and role guard on every request.
REQ-95-REQ-97	Auth/Data/Scheduler	Account deletion, audit logging and encrypted backup/restore.

The design references intentionally align with the design-reference terminology already used in the SRS traceability material (for example, dashboard query, recurring model, scheduler job, upload flow and RBAC matrix).

3.9 Security Architecture

PFEM treats financial information as sensitive. Security is implemented as a cross-cutting layer around every application request rather than as a feature of individual screens.

Threat / concern	Control
Spoofing / account takeover	Email verification, password policy, bcrypt hashing, JWT authentication and temporary lockout after repeated failures.
Tampering	Server-side validation, ownership checks, application-layer referential consistency and atomic financial operations.
Information disclosure	HTTPS, strict ownership filtering, RBAC, private attachment storage, signed URLs and non-sensitive error messages.
Denial of service / abuse	Rate limits/attempt limits around verification and authentication; bounded request processing and monitoring.
Elevation of privilege	Role guard and permission checks on every request; administrators do not receive another user's financial figures.
Auditability	Audit records for authentication events, permission failures, exports, bulk deletion and administrative actions; 12-month retention.
Data loss	Encrypted daily database backup, seven-day retention and restoration support.

Data isolation rule: every user-scoped query is constrained by the authenticated user identifier. Administrative access is limited to administrative functions and audit information specified by the SRS; it must not expose another user's financial figures.

4. Design

4.1 Design Overview

The detailed design follows the component boundaries established in Section 3. Controllers expose REST endpoints; services implement business rules; validators enforce input constraints; data-access modules isolate MongoDB operations; and adapters isolate external providers. Cross-cutting middleware handles authentication, authorization, error mapping and audit logging.

The principal domain entities are User, Account, Category, Transaction, Budget, RecurringRule, BillReminder, SavingsGoal, Notification, Attachment and AuditLog. These entities are stored as documents in MongoDB, with application-level referential consistency between related records.

4.1.1 Key Data Design Rules

- Monetary values are stored in a decimal-compatible representation; display uses Indian Rupee formatting where the base currency is INR.
- Dates are stored in UTC with original time-zone information; PFEM presents dates/times according to the SRS rules.
- Indexes exist on user identifier, user + transaction date, user + category, and user + account.
- Transfers are modeled so debit and credit sides are inseparable; partial transfers are not permitted.
- Deleted transactions remain recoverable for 30 days before scheduled permanent purge.
- Foreign-currency transactions persist the original amount, converted amount and exchange rate used so historical figures do not change retrospectively.

4.2 UML Sequence Diagrams

The following two flows satisfy the template requirement for at least two project-specific sequence diagrams and represent important end-to-end interactions.

4.2.1 User Authentication and Login

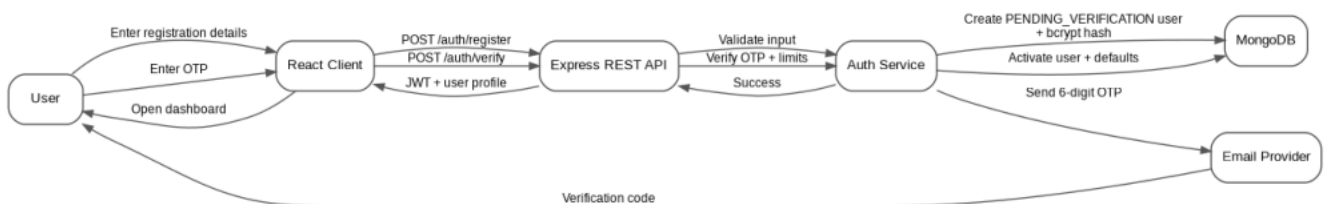


Figure 4.1 — Authentication sequence: client → REST API → authentication/session layer → data store, with email provider used for verification/recovery.

4.2.2 Transaction Entry

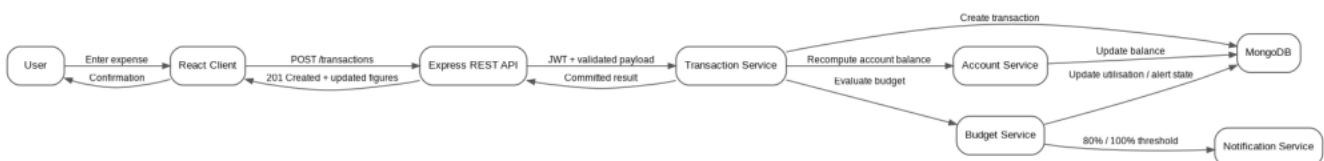


Figure 4.2 — Transaction sequence: client validation → authenticated API → transaction/account domain logic → persistence → dependent recomputation/notification.

4.3 API Design

The API is REST-oriented and served over HTTPS. The examples below define the principal contracts needed by the architecture; exact URI naming may be finalized during implementation while preserving the SRS behavior.

Endpoint	Method	Purpose	Representative request	Success / error
/api/auth/register	POST	Create account	name, email, password, mobile?	201; 400 validation / duplicate account
/api/auth/verify	POST	Verify email OTP	email, otp	200; 400 invalid/expired OTP
/api/auth/login	POST	Authenticate user	email, password	200 JWT; 401/423 authentication or lockout
/api/transactions	POST	Create transaction	type, account, amount, date, category, note...	201; 400 validation
/api/transactions/:id	PATCH	Edit transaction	changed transaction fields	200; 404/403/409
/api/transactions/:id	DELETE	Soft-delete transaction	confirmation	200; 404/403
/api/budgets	POST	Create budget	category, period, amount, rollover?	201; 400/409 overlap
/api/goals	POST	Create savings goal	name, target, date, account?	201; 400 validation
/api/reports	GET	Generate report	date range / report type	200; 400 invalid range
/api/import/csv	POST	Validate/preview CSV	multipart CSV	200 preview; 400 invalid file
/api/import/csv/commit	POST	Commit validated import	preview/import token	200 atomic result
/api/exports	GET	Export user data/report	format, date range	200 file; 403/404

Common API conventions

- Authorization: Bearer JWT for authenticated endpoints.
- Content type: application/json except multipart upload/import requests.
- Success responses contain the requested resource or operation result; error responses use a stable application error code and human-readable message.
- Server-side validation is authoritative even when equivalent client-side validation exists.
- Sensitive values such as passwords, verification codes and tokens are never returned in logs.

4.4 Error Handling, Logging & Monitoring

Area	Design
Validation errors	Return field-level errors; UI displays them beside the offending field.
Authentication errors	Use generic messages where revealing account existence would disclose information; lockout behavior follows the SRS.
Authorization errors	Return a controlled permission response without exposing internal identifiers or another user's data.
Business-rule errors	Return stable error codes/messages such as insufficient balance, overlapping budget or invalid date.
Unexpected errors	Map internal exceptions to a generic server-error response; never expose stack traces, SQL/DB errors or internal identifiers.

Area	Design
Audit logging	Record actor, action, timestamp and source address for authentication events, permission failures, exports, bulk deletion and administrative actions.
Operational monitoring	Monitor request latency, transaction failure rate, scheduler completion, notification delivery failures, import failures, backup status and error rate.

Performance observability is tied to the SRS targets: transaction save p95 ≤ 1 s, screen interactivity ≤ 2 s on 4G, dashboard ≤ 3 s, reports ≤ 5 s for the specified dataset, indexed queries ≤ 500 ms, and scheduler completion within the defined daily window.

4.5 UX Design

The UX layer follows the SRS UI requirements rather than defining a separate visual system in this SAD. The detailed wireframes/interface specification are maintained separately, as stated by the SRS.

UX concern	Design requirement carried into implementation
Navigation	Mobile bottom navigation: Dashboard, Transactions, Budgets, Reports and More; web uses the corresponding sidebar destinations.
Responsive layout	Correct rendering from 360 px to 1920 px without horizontal scrolling.
Money display	Indian Rupee symbol, two decimals and Indian digit grouping for INR.
Validation	Inline field-level errors with a corrective message; system errors use dismissible banners.
Long operations	Progress indicator when an action takes longer than one second.
Data-changing actions	Confirmation feedback and Undo where specified, especially deletion.
Empty states	Every list/chart provides guidance rather than an empty area or meaningless zero-value chart.
Accessibility	Minimum 4.5:1 text contrast, 44×44 px touch targets, keyboard navigation and text alternatives.
Theme	Light/dark theme with device setting followed by default.

4.6 Open Issues & Next Steps

Item	Status / next step
Exact cloud/provider selections	Finalize deployment provider and concrete email, push, object-storage and exchange-rate providers during implementation.
Mobile implementation framework	Select the implementation approach that satisfies the SRS mobile constraints while reusing the REST API.
API naming	Freeze final endpoint naming and OpenAPI specification before implementation sprint.
Database schema	Convert domain entities and SRS field layouts into validated MongoDB schemas and indexes.
Scheduler deployment	Define production scheduling mechanism and ensure only the test environment exposes the manual trigger endpoint.
Backup restoration	Perform and record a restoration test before final demonstration, as required by the SRS.
Performance validation	Benchmark against the SRS dataset assumptions and performance thresholds.
Future release features	Bank integrations, OCR receipt extraction, investment/portfolio tracking, tax computation, shared ledgers and multilingual support remain deferred.

5. Appendices

5.1 Glossary

Term	Meaning
Account	User-defined store of money such as cash, bank account, credit card or wallet.
Transaction	Expense, income or transfer recorded against an account.
Budget	Spending limit associated with a category/period or overall spending.
Recurring Rule	Definition used by the scheduler to create repeating transactions.
Bill Reminder	Scheduled notification for an upcoming payment.
Savings Goal	Target amount/date toward which contributions are recorded.
Net Worth	Assets less liabilities as defined by the SRS domain rules.
Audit Log	Security/administrative record of specified auditable events.
Soft Delete	Recoverable logical deletion before the 30-day purge.
RBAC	Role-based access control.

5.2 References

- PFEM Software Requirements Specification, Release 1.0 — project baseline.
- PFEM Test Plan — manual Unit, Integration and System test baseline.
- SAD Template — Software Architecture and Design Specification structure supplied for the project.
- IEEE Std 830-1998 — referenced by the PFEM SRS.
- OWASP Top 10 — 2021 Edition, referenced by the PFEM SRS.
- OWASP Mobile Application Security Verification Standard (MASVS), referenced by the PFEM SRS.
- Digital Personal Data Protection Act, 2023, referenced by the PFEM SRS.
- Information Technology Act, 2000 and Reasonable Security Practices Rules, 2011, referenced by the PFEM SRS.
- RFC 4180 — CSV format basis referenced by the PFEM SRS.

5.3 Architecture Decision Records (ADRs)

ADR	Decision	Rationale
ADR-01	Use three-tier layered architecture.	Matches the SRS product perspective and separates presentation, business logic and persistence.
ADR-02	Use a modular application tier rather than separately deployed microservices.	Reduces operational complexity while preserving clear service boundaries for a four-member project.
ADR-03	Use MongoDB as the document database.	Matches the SRS document-database requirement and supports the entity/document model.
ADR-04	Keep database access behind the application tier.	Centralizes validation, ownership checks, business rules and audit/security controls.
ADR-05	Use signed URLs for private receipt retrieval.	Prevents direct public access to financial attachments and limits retrieval lifetime.
ADR-06	Make scheduled financial operations idempotent.	Prevents duplicate recurring entries and supports recovery after scheduler interruptions.

5.4 Design-to-Test Alignment

The Test Plan contains 148 test cases: 64 Unit, 34 Integration and 50 System cases. It states that all functional requirements REQ-1 through REQ-97 and security item SEC-8 are represented at least once, with additional performance and safety checks. This SAD supplies the architecture/component boundaries that those tests exercise; execution results remain the responsibility of the Test Plan and are not duplicated here.

End of Software Architecture and Design Specification